

Classic Deep CNN Architectures

Naeemullah Khan

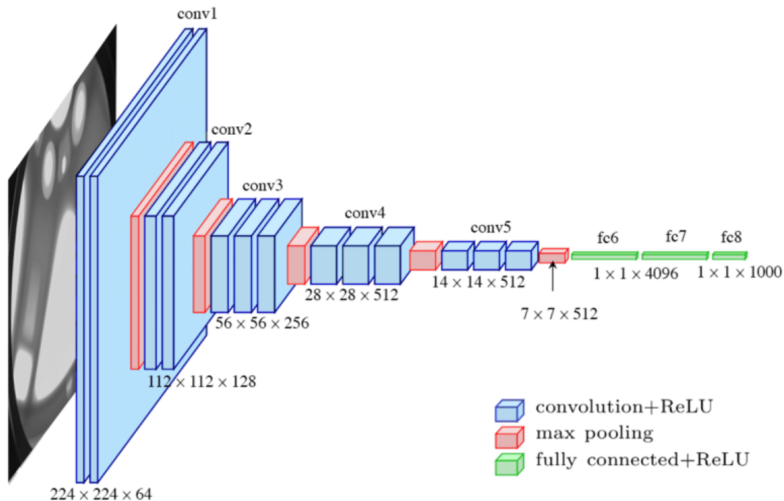
naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026



VGG-16 macroarchitecture; after D. Frossard, "VGG in TensorFlow" (redrawn).

1. Learning Outcomes
2. CNN Architectures
 1. LeNet-5
 2. AlexNet
 3. VGGNet
 4. InceptionNet
 5. ResNet
 6. EfficientNet
 7. MobileNet
3. ImageNet Dataset
4. Loss Functions & Optimizers
5. Limitations and Future Directions

6. References & Resources

By the end of this session, you will be able to:

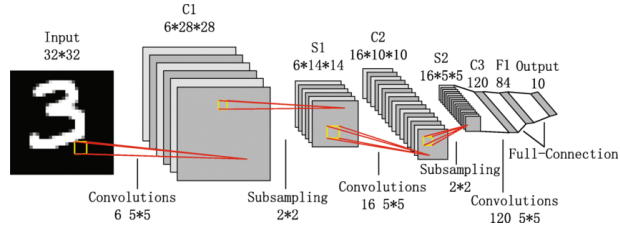
- ▶ See how deep CNNs have evolved over time.
- ▶ Get to know influential architectures: LeNet, AlexNet, VGG, InceptionNet, ResNet, EfficientNet, and MobileNet.
- ▶ Understand the key innovation behind each model (e.g., ReLU + dropout, small stacked filters, inception modules, skip connections, compound scaling, depthwise separable convolutions).
- ▶ Connect these architectures to the ImageNet benchmark that drove their development.
- ▶ Recall the common loss functions and optimizers used to train CNNs.
- ▶ Spot the limitations of CNNs and think about what's next.

CNN Architectures

Over time, researchers built advanced CNN architectures to improve performance and efficiency. These architectures introduced key innovations:

- ▶ **LeNet [LeCun et al., 1998]**: The first CNN trained end-to-end with backpropagation, designed for handwritten digit recognition.
- ▶ **AlexNet [Krizhevsky et al., 2012]**: The first CNN to achieve breakthrough performance on image classification.
- ▶ **VGGNet [Simonyan and Zisserman, 2014]**: Used very deep networks (up to 19 layers).
- ▶ **InceptionNet (GoogLeNet) [Szegedy et al., 2014]**: Used multiple filter sizes per layer (Inception modules).
- ▶ **ResNet [He et al., 2015]**: Introduced skip connections for training very deep networks.
- ▶ **EfficientNet [Tan and Le, 2019]**: Found a scaling method that simultaneously scales a CNN's depth, width, and resolution optimally using a single scaling coefficient.
- ▶ **MobileNet [Howard et al., 2017]**: Designed for mobile and embedded vision applications, using depthwise separable convolutions.

CNN Architectures: **LeNet-5**

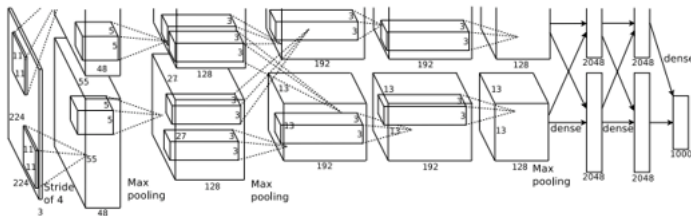


LeNet-5 architecture. Adapted from LeCun et al., 1998.

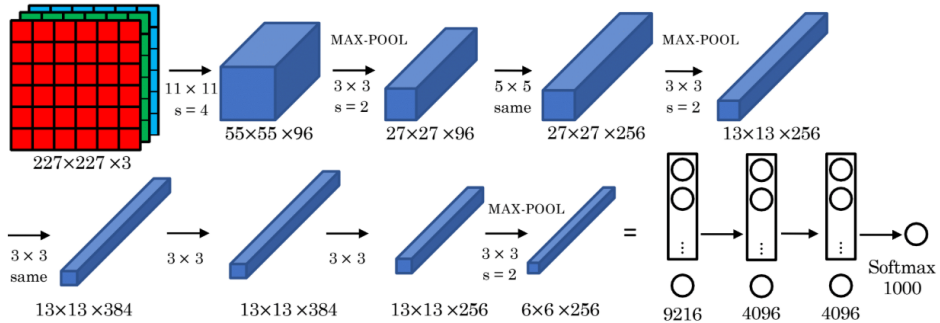
- ▶ LeNet-5 is a pioneering convolutional neural network (CNN) architecture developed by Yann LeCun and his collaborators; the LeNet series began in 1989 and culminated in LeNet-5 (LeCun et al., 1998).
- ▶ It was built to recognize handwritten digits, like those in the MNIST dataset.
- ▶ The network uses layers that look for patterns (convolution), shrink images (pooling), and finally make decisions (fully connected layers).
- ▶ LeNet-5 introduced ideas like using filters to find features and pooling to reduce size, which are now common in modern CNNs.

CNN Architectures: **AlexNet**

- ▶ First big improvement in image classification: won ILSVRC 2012 with 16.4% top-5 error, down from 25.8% the year before.
- ▶ Made use of convolutional layers, pooling, dropout, ReLU and training on GPUs.
- ▶ 5 convolutional layers, some followed by max-pooling layers, with three fully connected layers at the end.



Source: Krizhevsky et al., 2012.



[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Source: Andrew Ng, Deep Learning Specialization (deeplearning.ai).

CNN Architectures: **VGGNet**

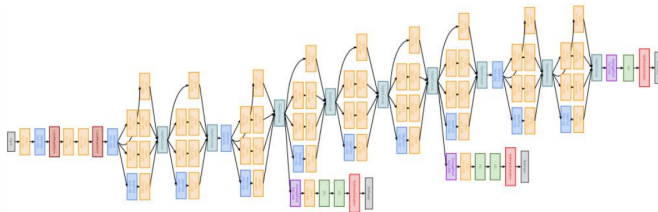
- **Improvement over AlexNet:** Uses a deeper network with small filters instead of a shallow network with larger filters.
- A stack of 3x3 conv layers (vs. 11x11 conv) has the same receptive field, more non-linearities, fewer parameters and deeper network representation.
- Two variants: VGG16 (13 conv + 3 FC layers) and VGG19 (16 conv + 3 FC layers).



Figure: Stanford CS231n, Lecture 9; configurations D and E of Simonyan & Zisserman, 2014 (Table 1).

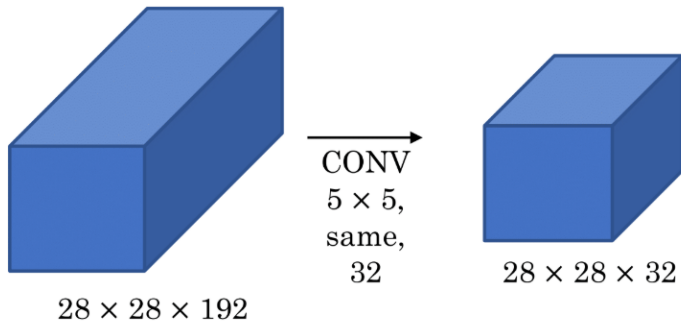
CNN Architectures: **InceptionNet**

- ▶ InceptionNet goes **deeper** with 22 layers.
- ▶ It is much more **efficient**: about 6.8 million parameters, far fewer than AlexNet's 60M.
- ▶ Uses a special building block called the **Inception module**.
- ▶ Adds **1x1 convolutions** (bottleneck layers) to make the network faster and smarter.
- ▶ Won the ILSVRC 2014 classification task with 6.7% top-5 error.



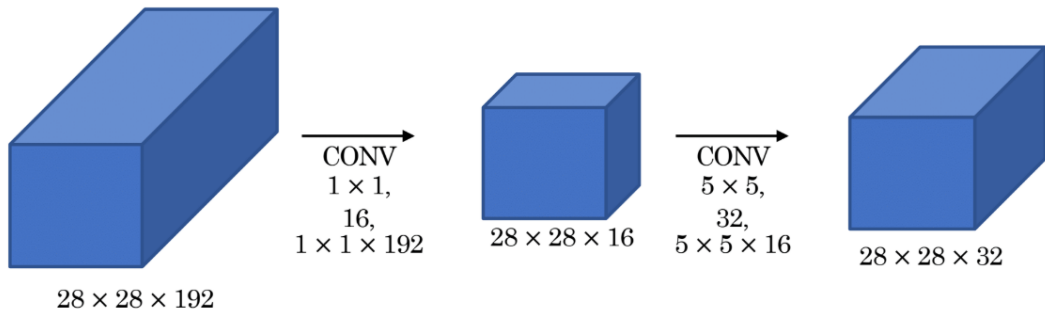
GoogLeNet architecture. Figure: Stanford CS231n, Lecture 9, after Szegedy et al., 2014, Fig. 3.

The problem of computational cost



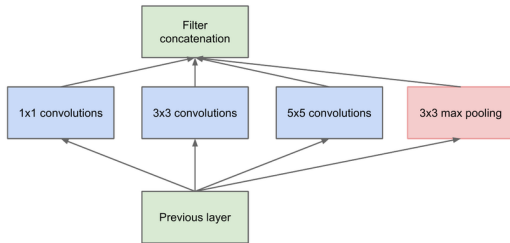
Source: Andrew Ng, Deep Learning Specialization (deeplearning.ai).

Using 1x1 convolution

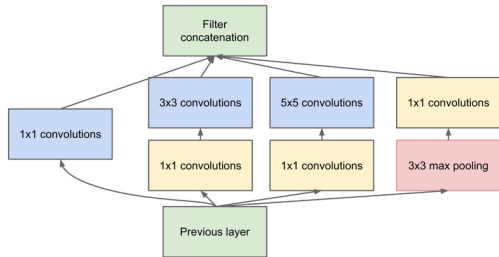


Source: Andrew Ng, Deep Learning Specialization (deeplearning.ai).

- **Inception module:** Runs 1×1 , 3×3 and 5×5 convolutions and a 3×3 max-pooling branch in parallel, then concatenates their outputs along the channel axis.
- The naive version (a) is expensive, so GoogLeNet uses (b): a 1×1 convolution shrinks the channel count before each 3×3 and 5×5 convolution, and after the pooling branch.

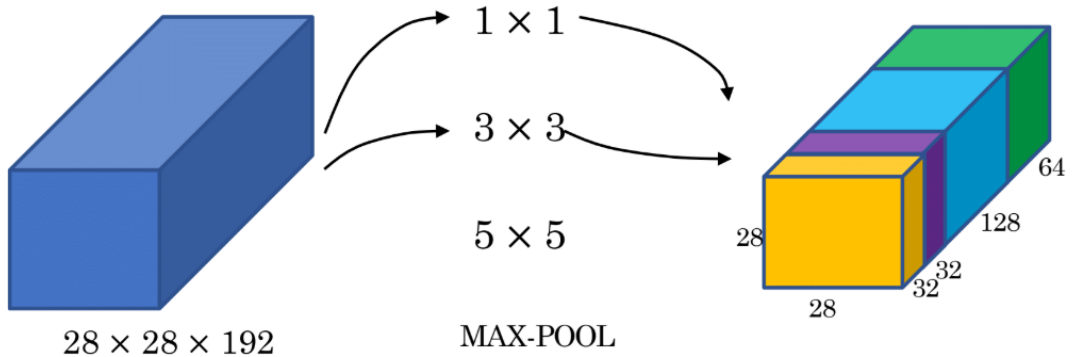


(a) Inception module, naïve version



(b) Inception module with dimension reductions

Source: Szegedy et al., "Going Deeper with Convolutions", CVPR 2015, Fig. 2.

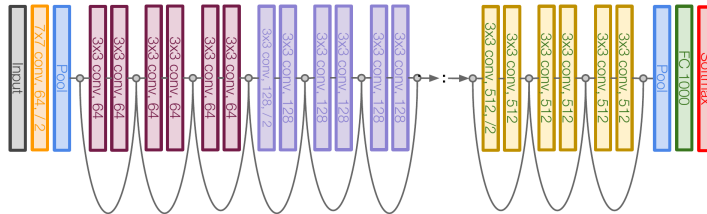


Source: Andrew Ng, Deep Learning Specialization (deeplearning.ai).

CNN Architectures: **ResNet**

- ▶ **Problem:** Making networks deeper does not always improve accuracy.
- ▶ **Why?** Deeper plain networks are harder to optimize, even though batch normalization keeps gradients from vanishing. He et al. call this the **degradation problem**.
- ▶ **Solution:** Residual Network (ResNet) introduces **skip connections (residuals)**, allowing information to flow more easily.

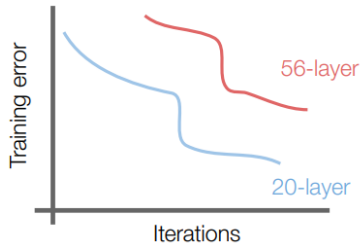
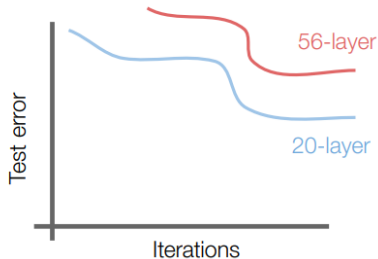
- ▶ Very deep networks using residual connections
- ▶ 152-layer model for ImageNet
- ▶ Stacked Residual Blocks
- ▶ **Residual:** A shortcut connection that helps the network pass information through layers more easily.
- ▶ Won ILSVRC 2015 with 3.57% top-5 error (ensemble).



ResNet-34. Figure: Stanford CS231n, Lecture 9, after He et al., 2015, Fig. 3.

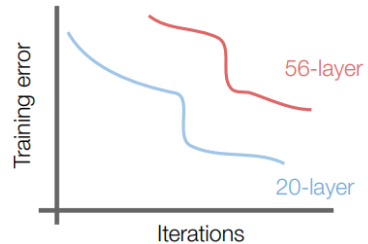
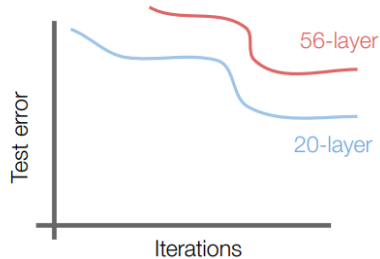
- ▶ What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

- What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



Test error (left) and training error (right) on CIFAR-10. Figure: Stanford CS231n, Lecture 9, after He et al., 2015, Fig. 1.

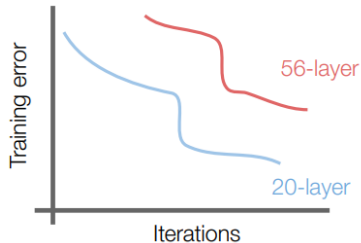
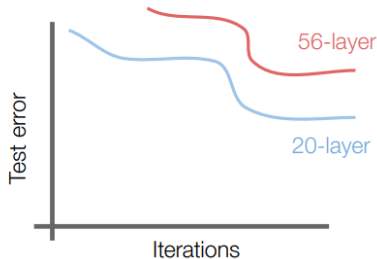
- What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



Test error (left) and training error (right) on CIFAR-10. Figure: Stanford CS231n, Lecture 9, after He et al., 2015, Fig. 1.

- 56-layer model shows higher error on both training and test sets

- What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



Test error (left) and training error (right) on CIFAR-10. Figure: Stanford CS231n, Lecture 9, after He et al., 2015, Fig. 1.

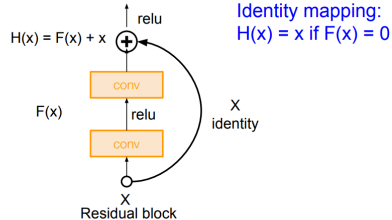
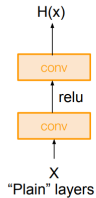
- 56-layer model shows higher error on both training and test sets
- The deeper model performs worse, but it's not caused by overfitting!

- **Fact:** Deep models have more representation power (more parameters) than shallower models.

- ▶ **Fact:** Deep models have more representation power (more parameters) than shallower models.
- ▶ **Hypothesis:** The problem is an optimization problem: deeper models are harder to optimize.

- ▶ **Fact:** Deep models have more representation power (more parameters) than shallower models.
- ▶ **Hypothesis:** The problem is an optimization problem: deeper models are harder to optimize.
- ▶ **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

- **Fact:** Deep models have more representation power (more parameters) than shallower models.
- **Hypothesis:** The problem is an optimization problem: deeper models are harder to optimize.
- **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



Plain layers vs. a residual block. Figure: Stanford CS231n, Lecture 9, after He et al., 2015, Fig. 2.

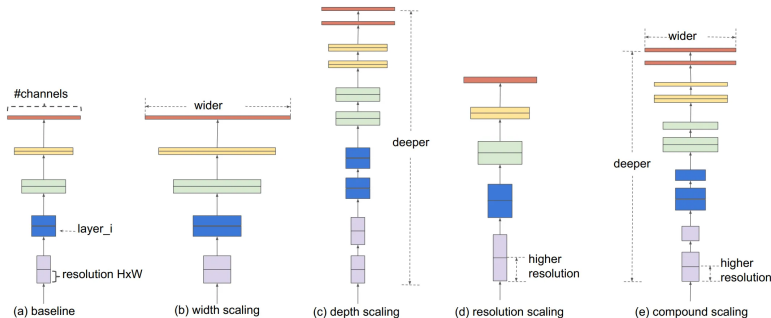
CNN Architectures: **EfficientNet**

► **Problem:** To improve accuracy, we can:

- Increase the number of layers (**depth**)
- Increase the number of neurons in each layer (**width**)
- Use higher-resolution images (**resolution**)

But finding the right balance of these three was largely based on trial and error.

► **Solution:** EfficientNet introduced **compound scaling**, a formula that systematically balances **depth**, **width**, and **resolution**.



EfficientNet Scaling. Source: Tan & Le, 2019 (EfficientNet, ICML).

- ▶ **Compound Scaling:** Uses a single **scaling coefficient** (ϕ) to control:
 - **Network Depth** (α^ϕ) $\rightarrow$ More layers
 - **Network Width** (β^ϕ) $\rightarrow$ More channels per layer
 - **Input Resolution** (γ^ϕ) $\rightarrow$ Larger input images
- ▶ The goal: find α, β, γ that balance accuracy & efficiency, then scale up optimally by increasing the global coefficient ϕ .

- ▶ EfficientNet optimizes depth, width, and resolution using this constraint:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

- ▶ **Why this equation?**

- Increasing **depth** (α) increases FLOPs **linearly**.
 - Increasing **width** (β) increases FLOPs **quadratically** (β^2).
 - Increasing **resolution** (γ) increases FLOPs **quadratically** (γ^2).
- ▶ To double total FLOPs, the three factors must be balanced together.

- ▶ The authors of EfficientNet searched for the best scaling factors on a small baseline model.
- ▶ They found:

$$\alpha = 1.2, \quad \beta = 1.1, \quad \gamma = 1.15$$

- The EfficientNet family (B0 to B7) is generated using:

$$\text{Depth} = 1.2^\phi, \quad \text{Width} = 1.1^\phi, \quad \text{Resolution} = 1.15^\phi$$

Model	ϕ	Depth (α^ϕ)	Width (β^ϕ)	Resolution (γ^ϕ)
B0	0	$1.2^0 = 1.0$	$1.1^0 = 1.0$	$1.15^0 = 1.0$
B1	1	$1.2^1 = 1.2$	$1.1^1 = 1.1$	$1.15^1 = 1.15$
B2	2	$1.2^2 = 1.44$	$1.1^2 = 1.21$	$1.15^2 = 1.32$
B3	3	$1.2^3 = 1.73$	$1.1^3 = 1.33$	$1.15^3 = 1.52$
B4	4	$1.2^4 = 2.07$	$1.1^4 = 1.46$	$1.15^4 = 1.75$
B5	5	$1.2^5 = 2.49$	$1.1^5 = 1.61$	$1.15^5 = 2.01$
B6	6	$1.2^6 = 2.99$	$1.1^6 = 1.77$	$1.15^6 = 2.31$
B7	7	$1.2^7 = 3.58$	$1.1^7 = 1.95$	$1.15^7 = 2.66$

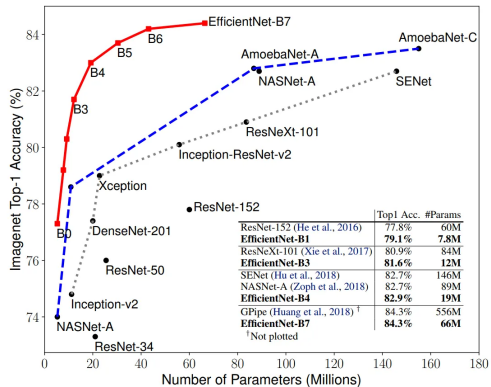
Scaling EfficientNet from B0 to B7

- We multiply these scaling factors by the baseline EfficientNet-B0 values and round to get the depth, width, and input resolution of each model: B0 is trained at 224×224 , B7 at 600×600 .

- ▶ EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.

- ▶ EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.
- ▶ EfficientNet-B7 reaches 84.3% Top-1 and 97.0% Top-5 accuracy on ImageNet.

- ▶ EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.
- ▶ EfficientNet-B7 reaches 84.3% Top-1 and 97.0% Top-5 accuracy on ImageNet.
- ▶ At that accuracy it is 8.4x smaller and 6.1x faster on inference than GPipe, the best ConvNet available at the time.



EfficientNet Performance on ImageNet. Source: Tan & Le, 2019 (EfficientNet, ICML).

CNN Architectures: **MobileNet**

► Why MobileNets?

- Small-sized models are crucial for mobile and embedded devices.
- MobileNets reduce computational cost and memory usage while maintaining good accuracy.

► Key Idea:

- Use **depthwise-separable convolutions** to significantly reduce computation compared to standard convolutions.

► Computational cost of standard convolution:

$$\text{Cost} = \# \text{ filter params} \times \# \text{ filter positions} \times \# \text{ filters}$$

- Filters operate on all input channels, increasing computation significantly.



Figure adapted from Andrew Ng, Deep Learning Specialization (deeplearning.ai).

- ▶ Split standard convolution into two steps:
 - **Depthwise Convolution:** Applies a single filter per input channel.
 - **Pointwise Convolution:** Combines outputs from depthwise convolution.
- ▶ **Key Benefit:** Reduces computational cost significantly compared to standard convolution.

Depthwise-Separable Convolutions (cont.)

Normal Convolution



Depthwise Separable Convolution

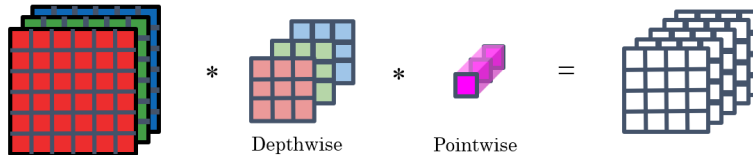
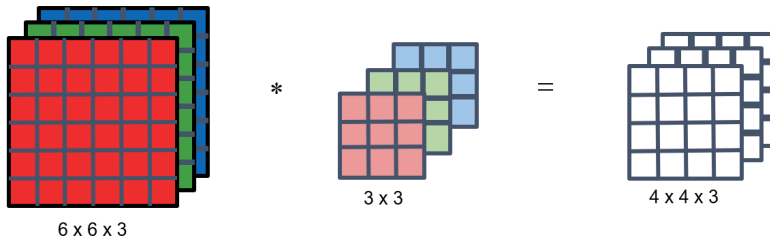


Figure adapted from Andrew Ng, Deep Learning Specialization (deeplearning.ai).

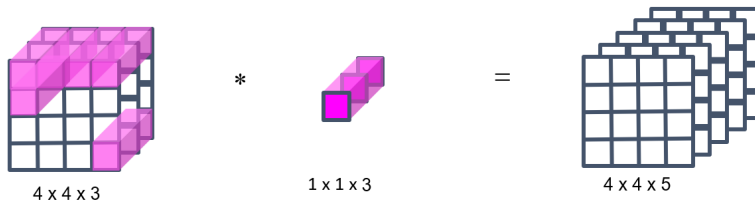
- Operates on each input channel separately.



Computational cost = #filter params x # filter positions x # of filters

Figure adapted from Andrew Ng, Deep Learning Specialization (deeplearning.ai).

- Combines outputs from depthwise convolution using 1×1 convolutions (mixes channels).



$$\text{Computational cost} = \# \text{filter params} \times \# \text{filter positions} \times \# \text{of filters}$$

Figure adapted from Andrew Ng, Deep Learning Specialization (deeplearning.ai).

- Write D_K for the kernel size, D_F for the output feature-map size, M for the number of input channels and N for the number of output channels.

Standard convolution: $D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F$

Depthwise + pointwise: $D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F$

- Dividing the second by the first gives the cost ratio $\frac{1}{N} + \frac{1}{D_K^2}$, so the saving grows with the number of output channels.
- With 3×3 kernels MobileNet needs 8 to 9 times less computation than a standard convolution, at a small cost in accuracy [Howard et al., 2017].

- ▶ **MobileNet v1** [Howard et al., 2017. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*]:
 - Replaces the standard convolutions of a plain CNN with depthwise-separable blocks.
 - 4.2M parameters and 70.6% top-1 accuracy on ImageNet.
- ▶ **MobileNet v2** [Sandler et al., 2018. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*]:
 - Adds **residual connections**.
 - Introduces:
 - ▶ **Expansion step**: Expands input dimensions before depthwise convolution.
 - ▶ **Projection step**: Reduces dimensions after processing.

MobileNet v1



MobileNet v2

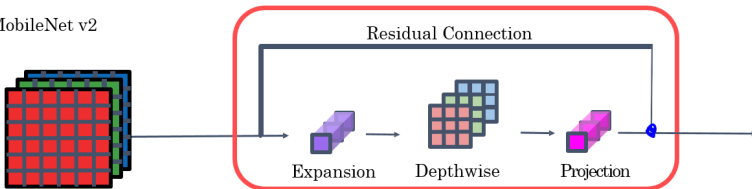


Figure adapted from Andrew Ng, Deep Learning Specialization (deeplearning.ai).

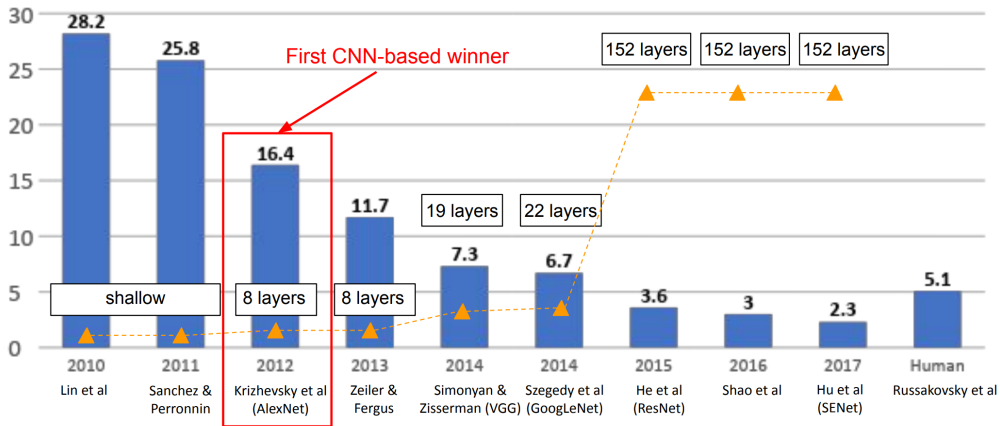
Model	Year	Key Idea	#Params	Strength
LeNet-5	1998	Early CNN for digit recognition	~60K	Pioneered CNNs
AlexNet	2012	Deeper + ReLU + Dropout	~60M	Started Deep Learning
VGG	2014	3×3 filters, uniform design	~138M	Simplicity
Inception	2014	Multi-scale filters	~6.8M	Efficient
ResNet	2015	Skip connections	~25M (50)	Very deep models
EfficientNet	2019	Compound scaling	~5M (B0)	Accuracy/efficiency trade-off
MobileNet	2017	Lightweight architecture	~4M	Mobile-ready

CNN Datasets: **ImageNet**

- ▶ The most extensive dataset for image classification
- ▶ 3 RGB channels from 0 to 255
- ▶ Full ImageNet: 14,197,122 images across 21,841 categories
- ▶ ILSVRC benchmark subset (used by the architectures in this lecture): 1000 classes, about 1.28M training images



Mosaic of ImageNet sample images; original source unidentified.



Winning top-5 classification error by year. Figure: Stanford CS231n; data from the official ILSVRC results (image-net.org).

CNN: Loss Functions & Optimizers

Loss:

- ▶ Loss functions measure how well the model's predictions match the true labels.
- ▶ Common loss functions include:
 - Cross-Entropy Loss (for classification)
 - Mean Squared Error (for regression)
- ▶ The choice of loss function depends on the task at hand.

Optimizers:

- ▶ Optimizers update the model's parameters based on the gradients computed during backpropagation.
- ▶ Common optimizers include:
 - Stochastic Gradient Descent (SGD)
 - Adam
 - RMSprop
- ▶ The choice of optimizer can significantly affect the training speed and convergence.

Code snippet (PyTorch)

```
import torch.nn as nn
import torch

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss = criterion(preds, labels)
loss.backward()
optimizer.step()
```

CNN: Limitations and Future Directions

- ▶ **Understanding global context:** CNNs are inherently local due to convolutional filters, making it hard to capture long-range dependencies.
- ▶ **Variations outside training data:** Performance drops significantly when faced with data distributions not seen during training.
- ▶ **Transfer learning limitations:** Pre-trained CNNs do not always generalize well to new tasks or domains.
- ▶ **BatchNorm sensitivity:** Batch Normalization effectiveness depends on batch size, which can be problematic for small-batch or online learning.
- ▶ **Compute constraints:** Deploying CNNs on mobile or edge devices remains challenging due to high computational and memory requirements.

- ▶ **Vision Transformers (ViTs):** Using self-attention to capture global context and long-range dependencies.
- ▶ **Self-supervised learning:** Reducing reliance on labeled data by learning useful representations from unlabeled data.
- ▶ **More efficient models:** Architectures like EfficientNet optimize accuracy and efficiency for deployment on resource-constrained devices.
- ▶ **Better augmentation:** Automated data augmentation techniques (e.g., AutoAugment, RandAugment) improve generalization.
- ▶ **Low-power inference:** Techniques such as quantization and pruning enable CNNs to run efficiently on edge and mobile devices.

CNN: References & Resources

Books:

- ▶ **Deep Learning** by Ian Goodfellow, Yoshua Bengio, and Aaron Courville
- ▶ **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow** by Aurélien Géron
- ▶ **Pattern Recognition and Machine Learning** by Christopher Bishop
- ▶ **Deep Learning for Computer Vision with Python** by Adrian Rosebrock

Online Courses:

- ▶ **Deep Learning Specialization** by Andrew Ng (Coursera)
- ▶ **Convolutional Neural Networks for Visual Recognition** by Fei-Fei Li (Stanford University)
- ▶ **Practical Deep Learning for Coders** by Jeremy Howard (fast.ai)
- ▶ **Deep Learning with PyTorch** by Facebook AI Research

Research Papers:

- ▶ **Gradient-Based Learning Applied to Document Recognition** by Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner (Proceedings of the IEEE, 1998)
- ▶ **ImageNet Classification with Deep Convolutional Neural Networks** by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton (NIPS 2012)
- ▶ **Very Deep Convolutional Networks for Large-Scale Image Recognition** by K. Simonyan and A. Zisserman (ICLR 2015)
- ▶ **Going Deeper with Convolutions** by Christian Szegedy et al. (CVPR 2015)
- ▶ **Deep Residual Learning for Image Recognition** by Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (CVPR 2016)

Research Papers (cont.):

- ▶ **MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications** by Andrew G. Howard et al. (arXiv:1704.04861, 2017)
- ▶ **MobileNetV2: Inverted Residuals and Linear Bottlenecks** by Mark Sandler et al. (CVPR 2018)
- ▶ **EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks** by Mingxing Tan and Quoc V. Le (ICML 2019)
- ▶ **ImageNet Large Scale Visual Recognition Challenge** by Olga Russakovsky et al. (IJCV, 2015)

Websites and Blogs:

- ▶ **Towards Data Science** (Medium)
- ▶ **Distill.pub** (Research and visualization)
- ▶ **Kaggle** (Datasets and competitions)
- ▶ **arXiv** (cs.CV preprints, including every paper listed above)

YouTube Channels:

- ▶ **3Blue1Brown** (Mathematics and deep learning)
- ▶ **Two Minute Papers** (Research summaries)
- ▶ **Sentdex** (Python programming and machine learning)
- ▶ **StatQuest with Josh Starmer** (Statistics and machine learning)

GitHub Repositories:

- ▶ **TensorFlow Models** (TensorFlow implementations)
- ▶ **PyTorch Examples** (PyTorch implementations)
- ▶ **fastai** (High-level library for deep learning)
- ▶ **Keras** (High-level neural networks API)

Conferences:

- ▶ **NeurIPS** (Neural Information Processing Systems)
- ▶ **CVPR** (Computer Vision and Pattern Recognition)
- ▶ **ICML** (International Conference on Machine Learning)
- ▶ **ICLR** (International Conference on Learning Representations)